

HavenWorld Build Guide

Scaling Beyond Free Tier, Horizontal Architecture, WebGPU & Database at Scale

Author: Daniel | **Date:** Friday, 18 September 2026 | **Location:** Anoka, MN, United States
| **Series Part:** 10A / 10

Project: HavenWorld — Browser-Based 3D Social Virtual World (Club Penguin / Habbo Hotel style, solo indie dev)

Series Context — Read Before Implementing

This guide covers **future scaling steps**. Implement these only when the free tier shows measurable strain. Premature scaling adds operational overhead, cost, and complexity with zero player benefit. Monitor your metrics first — act on data, not anxiety.

Current Tech Stack Reminder (as of Part 10A)

Layer

Technology

Notes

Hosting

Oracle Cloud Free Tier ARM VM

4 OCPUs, 24 GB RAM — Ampere A1

Frontend Renderer

Babylon.js 7 + Vite 6

WebGL2 now; WebGPU upgrade in Part 10A

Backend Runtime

Node.js 22 LTS + Express 5 + Socket.io 4

PM2 cluster mode, 4 workers

ORM + Database

Prisma ORM + PostgreSQL 16

Hosted on Oracle VM

Cache / Pub-Sub

Redis 7

Socket.io adapter + game state cache

CDN / Frontend Host

Cloudflare Pages

havenworld.pages.dev

Asset Storage

Cloudflare R2

10 GB/month free egress

DNS

nip.io free DNS

No custom domain required

Series Progress

Table of Contents

1. **When Oracle Free Tier Isn't Enough** — Warning signs, migration paths, full runbook
 2. **Horizontal Scaling: Multiple Node.js Instances** — PM2, Redis pub/sub, second VM, PgBouncer
 3. **WebGPU Readiness with Babylon.js 7** — Feature detection, config swap, A/B testing, monitoring
 4. **Database at Scale** — Partitioning, read replicas, query optimization, Redis strategy
 5. **Technical Debt Roadmap** — TypeScript strict mode, ESLint hardening, Dependabot, sprint template
 6. **Closing & Summary** — Thresholds table, forward link to Part 10B
-

Section 1 — When Oracle Free Tier Isn't Enough

The Oracle Cloud Always Free tier ARM VM (4 OCPUs, 24 GB RAM) is remarkably capable for a solo indie game. Most HavenWorld deployments will run comfortably on this tier for hundreds of concurrent players. However, sustained organic growth will eventually surface bottlenecks. This section defines exactly when and how to act.

1.1 Warning Signs Checklist

Before taking any scaling action, confirm that **two or more** of the following warning signs appear simultaneously and persist for more than one hour. A single spike is not a scaling trigger — investigate the root cause first.

```
htop vmstat 5 20    # 20 samples, 5-second intervals
free -h
SELECT query, mean_exec_time, calls FROM pg_stat_statements WHERE
mean_exec_time > 200 ORDER BY mean_exec_time DESC LIMIT 20;
// apps/server/src/routes/health.ts const start = process.hrtime.bigint();
setImmediate(() => {    const lag = Number(process.hrtime.bigint() - start) /
1_000_000;    res.json({ status: 'ok', eventLoopLagMs: lag.toFixed(2) }); });
redis-cli info memory | grep -E "used_memory_human|maxmemory_human"
```

Tip — Set Up Alerts Before You Need Them

Configure a simple cron-based monitor that writes metric snapshots to a `system_health` PostgreSQL table every 5 minutes. This gives you a historical graph when diagnosing incidents rather than guessing in real time.

1.2 Migration Path Ranked by Cost (Free Tier First)

If warning signs are confirmed, choose the cheapest option that resolves the bottleneck. **Do not jump to a managed cloud provider unnecessarily** — most issues can be resolved within Oracle before paying anything.

Option	Monthly Cost	RAM	CPU	Best For	Recommendation
Oracle Cloud Always Free (current)	\$0	24 GB	4 OCPU Ampere A1	Everything up to ~500 CCU	✅ Stay here as long as possible
Oracle Ampere A1 Paid Shape Upgrade	~\$15/mo	Up to 96 GB	Up to 80 OCPU (beyond free 4)	CPU/RAM vertical scale on existing VM	🔴 First paid upgrade — cheapest path
Hetzner CX22 (EU, AMD x86)	~€4/mo (~\$4.40)	4 GB	2 vCPU AMD	Second VM for horizontal scaling	🔴 Best value for a second node

Option	Monthly Cost	RAM	CPU	Best For	Recommendation
Hetzner CAX11 (EU, ARM)	~€4.15/mo (~\$4.60)	4 GB	2 vCPU Ampere ARM	ARM-native second node (matches Oracle stack)	🔴 Preferred for ARM consistency
Fly.io Machines	~\$5-\$10/mo	256 MB-1 GB per machine	Shared, pay-per-use	Stateless microservices only	⚠️ Not suitable for Socket.io stateful workers
Railway Hobby Plan	\$5/mo	8 GB	8 vCPU shared	Rapid experimentation	⚠️ Higher cost per GB than Hetzner
Render Starter	\$7/mo	512 MB	Shared	Simple REST APIs only	❌ Insufficient RAM for HavenWorld server



Recommendation

Stay on Oracle Always Free as long as possible. The free 4 OCPUs and 24 GB RAM are more capable than any \$5/month competitor. When you genuinely outgrow it, the **first upgrade path** is adding RAM and CPU within Oracle's paid Ampere A1 shapes (cheapest cents-per-GB-RAM on the market). For true geographic redundancy or a second node, **Hetzner CAX11 (ARM)** is the best-value option to mirror your Oracle stack.

1.3 Full Data Migration Script (Oracle → Hetzner or Any New VM)

Follow this runbook when migrating to a new server. Complete every step in order. Do not skip the smoke-test checklist at the end.

Step 1 — Full PostgreSQL Backup

[LINUX] Run on the **source VM** (Oracle). The `-Fc` flag produces a compressed custom-format dump compatible with `pg_restore`:

```
pg_dump -Fc havenworld > havenworld_backup_$(date +%Y%m%d).dump ls -lh
havenworld_backup_*.dump # confirm file size looks reasonable
```

Step 2 — Compress and Transfer to New Server

[LINUX] Use `rsync` over SSH for efficient, resumable transfer. Replace `NEW_SERVER_IP` with the target VM's IP address:

```
rsync -avz --progress \ havenworld_backup_$(date +%Y%m%d).dump \
ubuntu@NEW_SERVER_IP:/home/ubuntu/havenworld_restore/
```

Step 3 — Restore Database on New Server

[LINUX] Run on the **destination VM**. Create the database first, then restore.

Replace `YYYYMMDD` with your actual dump date:

```
createdb havenworld pg_restore -d havenworld \
/home/ubuntu/havenworld_restore/havenworld_backup_YYYYMMDD.dump # Verify row
counts match source psql -d havenworld -c "SELECT schemaname, tablename,
n_live_tup FROM pg_stat_user_tables ORDER BY n_live_tup DESC;"
```

Step 4 — Migrate Redis Data

[LINUX] Trigger an RDB snapshot on the source, then copy and restore it. Redis data is ephemeral game state, so this step is optional — Redis will warm up naturally from PostgreSQL on first load:

```
# On SOURCE VM — trigger a blocking save redis-cli BGSAVE # Wait for
"Background saving terminated with success" redis-cli LASTSAVE # Copy the
dump file to new server rsync -avz /var/lib/redis/dump.rdb
ubuntu@NEW_SERVER_IP:/tmp/dump.rdb # On DESTINATION VM — stop Redis, replace
dump, restart sudo systemctl stop redis sudo cp /tmp/dump.rdb
/var/lib/redis/dump.rdb sudo chown redis:redis /var/lib/redis/dump.rdb sudo
systemctl start redis redis-cli PING # expect: PONG
```

Step 5 — Configure Environment Variables on New Server

[LINUX] Copy your `.env` from the source and update the connection strings to point to the new server's local services:

```
# Copy .env from source (or recreate it) rsync -avz
ubuntu@OLD_SERVER_IP:/opt/havenworld/apps/server/.env \
/opt/havenworld/apps/server/.env # Edit DB_URL and REDIS_URL to point to
localhost on new server nano /opt/havenworld/apps/server/.env #
DATABASE_URL="postgresql://havenworld_user:PASSWORD@127.0.0.1:5432/
havenworld" # REDIS_URL="redis://127.0.0.1:6379"
```

Step 6 — Update Nginx Config with New Server IP

[LINUX] On the new server, install Nginx and configure it identically to your Oracle setup. Update the `nip.io`-based `server_name` to reflect the new IP:

```
sudo nano /etc/nginx/sites-available/havenworld # Update server_name to:
NEW_SERVER_IP.nip.io (e.g., 140-238-11-99.nip.io) sudo nginx -t # test
config syntax sudo systemctl reload nginx
```

Step 7 — Re-Run Certbot for New nip.io Subdomain

[LINUX] Each new IP address requires a fresh Let's Encrypt certificate against the new nip.io subdomain. Replace the IP below with your actual new server IP:

```
sudo certbot --nginx -d 140-238-11-99.nip.io \ --non-interactive --agree-tos -m daniel@example.com sudo systemctl reload nginx
```

Step 8 — Update Cloudflare Pages Environment Variables

[MAC] In the Cloudflare Pages dashboard, navigate to **havenworld.pages.dev** → **Settings** → **Environment Variables** and update the production variable:

```
VITE_API_URL=https://140-238-11-99.nip.io
```

Trigger a new Cloudflare Pages deployment to pick up the new variable. This can also be done via Wrangler CLI:

```
pnpx wrangler pages secret put VITE_API_URL --project-name havenworld
```

Step 9 — Run Prisma Migrations on New Server

[LINUX] Apply any pending migrations to the newly restored database to ensure the schema is at the latest version:

```
cd /opt/havenworld pnpm prisma migrate deploy pnpm prisma db seed # only if seed data is needed
```

Step 10 — Smoke Test Checklist

Before shutting down the old server, verify all critical paths on the new server:

- Authentication: register a test account and confirm JWT is returned
- Socket.io: open browser dev tools, confirm WebSocket upgrade (101 Switching Protocols)
- Room Join: enter a room and confirm player list populates via Socket.io events
- Cloudflare R2 asset load: confirm 3D assets load from R2 (check Network tab for R2 domain)
- Nginx HTTPS: confirm green padlock and valid cert on new nip.io domain
- Redis: confirm chat and room state works (pub/sub active)
- PM2: run `pm2 list` and confirm all 4 workers are *online*

Zero-Downtime Strategy

Keep the **old Oracle VM running** until the new server's smoke tests pass completely and DNS has propagated (usually 1–5 minutes for Cloudflare Pages env var redeployment). Only then power down the old VM. Because nip.io resolves to an IP address directly, there is no TTL-based DNS delay — the switch is instant once Cloudflare Pages redeploys.

Section 2 — Horizontal Scaling: Running Multiple Node.js Instances

Horizontal scaling for HavenWorld means running multiple Node.js server processes — either as workers on a single VM via PM2 cluster mode, or as separate processes on multiple VMs behind a load balancer. Both approaches require Redis as the coordination layer for Socket.io events.

2.1 PM2 Cluster Mode (Already Configured — Review)

PM2 cluster mode is the **zero-cost first step** in horizontal scaling. It forks multiple Node.js processes from a single VM, fully utilizing all available CPU cores. On the Oracle Free Tier ARM VM with 4 OCPUs, PM2 will spawn 4 worker processes.

Your `ecosystem.config.cjs` should already contain the following configuration from earlier guide parts:

```
// ecosystem.config.cjs module.exports = {  apps: [    {      name:      'havenworld-server',      script: './apps/server/dist/index.js',      instances: 'max',      // spawns one worker per available CPU core      exec_mode: 'cluster',      // Node.js cluster module – shared port      watch: false,      env_production: {        NODE_ENV: 'production',      },      max_memory_restart: '1500M', // restart worker if it exceeds 1.5 GB RAM      error_file: './logs/pm2-error.log',      out_file: './logs/pm2-out.log',    },  ],};
```

[LINUX] Confirm that all 4 workers are running and healthy:

```
pm2 list pm2 monit # real-time per-worker CPU/RAM dashboard
```

Expected output of `pm2 list` — you should see 4 rows with *status: online* and incrementing PIDs:

memory	id	name	mode	id	status	cpu
	0	havenworld-server	cluster	0	online	0%
210 MB	1	havenworld-server	cluster	0	online	0%
198 MB	2	havenworld-server	cluster	0	online	0%
204 MB	3	havenworld-server	cluster	0	online	0%
195 MB						

2.2 Socket.io Redis Pub/Sub Across Workers

Without the Redis adapter, Socket.io workers are **isolated**. A player connected to worker-0 cannot receive events emitted by a player on worker-2. The

`@socket.io/redis-adapter` package solves this by routing all events through Redis pub/sub channels, making all workers appear as a single logical Socket.io server.

Your existing configuration in `apps/server/src/index.ts` should already include this setup:

```
// apps/server/src/index.ts import { Server } from 'socket.io'; import
{ createAdapter } from '@socket.io/redis-adapter'; import { createClient }
from 'redis'; const pubClient = createClient({ url:
process.env.REDIS_URL }); const subClient = pubClient.duplicate(); // Redis
requires separate connections for pub and sub await
Promise.all([pubClient.connect(), subClient.connect()]);
io.adapter(createAdapter(pubClient, subClient)); console.log('[HavenWorld]
Socket.io Redis adapter active – all workers synchronized');
```

⚠ Why This Is Non-Negotiable

Without the Redis adapter, `io.emit('ROOM_UPDATE', data)` on worker-2 will only reach clients connected to worker-2. Players on worker-0, 1, and 3 will receive no event. This produces invisible-player bugs, missed chat messages, and stale room state — all of which are extremely difficult to reproduce in development. The Redis adapter eliminates this entire class of bugs.

2.3 Running a Second VM (True Horizontal Scaling)

When a single VM's 4 OCPUs are genuinely saturated (confirmed by sustained CPU >80%), adding a second VM provides true horizontal scaling. Both VMs share the same PostgreSQL and Redis instances, coordinated by Nginx acting as a reverse proxy load balancer.

[LINUX] On VM-2, install the identical stack as VM-1 (Node.js 22 LTS, pnpm, PM2, same codebase via git pull from your repository), then configure it to point to VM-1's shared services:

```
# On VM-2: configure .env to point to VM-1's PostgreSQL and Redis
DATABASE_URL="postgresql://havenworld_user:PASSWORD@129.146.88.42:5432/havenworld"
REDIS_URL="redis://129.146.88.42:6379" # Start PM2 on VM-2
pm2 start ecosystem.config.cjs --env production pm2 save
```

PostgreSQL Remote Access Warning

If VM-2 connects to PostgreSQL on VM-1 remotely, you must: (1) set `listen_addresses = '*'` in `postgresql.conf` on VM-1, (2) add a `pg_hba.conf` entry allowing the VM-2 IP, and (3) open port 5432 in Oracle's VCN Security List for that specific IP only. Never expose PostgreSQL to the public internet.

[LINUX] On VM-1 (or a dedicated Nginx load balancer), configure an upstream block with `ip_hash` for sticky sessions. Sticky sessions ensure a given client always reaches the same worker, which is important for Socket.io upgrade negotiation:

```
# /etc/nginx/sites-available/havenworld upstream havenworld_backend
{
    ip_hash; # sticky sessions – same client IP
    always routes to same upstream
    server 129.146.88.42:3001; # VM-1 – Oracle Free Tier ARM
    server 140.238.11.99:3001; # VM-2 – Hetzner CAX11 ARM
    keepalive 32; # maintain persistent connections to upstream workers
}
server {
    listen 443 ssl;
    server_name 129-146-88-42.nip.io;
    location / {
        proxy_pass http://havenworld_backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 86400s; # keep Socket.io connections alive
    }
}
```

[LINUX] Test and reload Nginx after editing:

```
sudo nginx -t && sudo systemctl reload nginx
```

Cloudflare Load Balancing (Optional)

Cloudflare's free tier supports health-check-based routing between 2 origins via their Load Balancing product (note: Load Balancing starts at \$5/month as of 2026 — verify pricing in the Cloudflare dashboard). For zero-cost two-origin routing, Nginx `ip_hash` upstream as shown above is sufficient. The Nginx

approach also gives you more granular sticky-session control for Socket.io long-polling fallback.

2.4 PgBouncer Connection Pooling

Each Node.js worker maintains a pool of PostgreSQL connections via Prisma. With 4 workers on VM-1 and 4 workers on VM-2, you may have 40–80 simultaneous connections to PostgreSQL — approaching PostgreSQL’s default `max_connections = 100` limit. PgBouncer is a lightweight connection pooler that multiplexes many application connections into a small number of actual database connections.

[LINUX] Install PgBouncer on the same VM as PostgreSQL:

```
sudo apt update
sudo apt install pgbouncer -y
pgbouncer --version # confirm installation
```

[LINUX] Configure PgBouncer at `/etc/pgbouncer/pgbouncer.ini`:

```
[databases] ; Forward connections to the local PostgreSQL instance havenworld
= host=127.0.0.1 port=5432 dbname=havenworld [pgbouncer] listen_port = 6432
; PgBouncer listens on 6432; PostgreSQL remains on 5432 listen_addr =
127.0.0.1 ; bind to localhost only – never expose externally auth_type =
md5 auth_file = /etc/pgbouncer/userlist.txt pool_mode = transaction ;
IMPORTANT: use transaction mode for Prisma compatibility max_client_conn =
200 ; maximum client connections PgBouncer will accept
default_pool_size = 20 ; actual connections forwarded to PostgreSQL
server_idle_timeout = 60 ; release idle server connections after 60s
log_connections = 1 log_disconnections = 1
```

[LINUX] Add the database user to PgBouncer’s auth file. The password must be MD5 hashed (`md5 + md5(password + username)`):

```
# Generate the md5 hash echo -n "PASSWORDhavenworld_user" | md5sum # Add to
userlist.txt in format: "username" "md5HASH" sudo nano
/etc/pgbouncer/userlist.txt # "havenworld_user" "md5a3f2b1c4d5e6f7..."
```

[LINUX] Update `DATABASE_URL` in your server’s `.env` to route through PgBouncer’s port:

```
# .env – change port from 5432 to 6432
DATABASE_URL="postgresql://havenworld_user:PASSWORD@127.0.0.1:6432/
havenworld"
```

[LINUX] Enable and start PgBouncer:

```
sudo systemctl enable pgbouncer
sudo systemctl start pgbouncer
sudo systemctl status pgbouncer # confirm active (running)
```

 **Transaction Pool Mode vs Session Pool Mode — Prisma Compatibility**

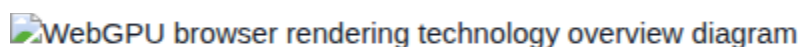
Transaction pool mode (`pool_mode = transaction`): A database connection is held only for the duration of a single transaction, then returned to the pool. This is the most efficient mode and is **required for Prisma**. However, it is incompatible with PostgreSQL features that rely on session-level state, such as `SET LOCAL`, advisory locks, and `LISTEN/NOTIFY`.

Session pool mode (`pool_mode = session`): A connection is held for the entire client session lifetime. Safe for all PostgreSQL features but provides less connection multiplexing benefit.

Use transaction mode with Prisma. Prisma does not use session-level state. If you use raw `LISTEN/NOTIFY` for real-time features, create a separate direct connection to PostgreSQL on port 5432 for those specific use cases.

Section 3 — WebGPU Readiness with Babylon.js 7

WebGPU is the next-generation web graphics API, replacing WebGL as the primary bridge between JavaScript and the GPU. Babylon.js 7 ships with a production-ready WebGPU renderer that is feature-complete with the WebGL2 renderer, making the transition straightforward for HavenWorld.



WebGPU provides direct, low-overhead access to GPU hardware from the browser — enabling compute shaders, faster command submission, and better parallelism than WebGL.

3.1 What WebGPU Brings to HavenWorld

WebGPU is not merely a visual upgrade — it is an architectural shift in how the browser communicates with the GPU. For HavenWorld, the practical benefits include:

- **Faster GPU command submission.** WebGL serializes all draw calls through a CPU-side validation layer. WebGPU pre-compiles command buffers on the

GPU, reducing per-frame CPU overhead by 30–50% in command-heavy scenes.

- **Compute shaders for particle effects and physics pre-computation.** HavenWorld’s weather system (rain, snow), fishing bobber physics, and avatar cloth simulation can be offloaded to GPU compute shaders via Babylon.js’s `ComputeShader` API — freeing the CPU event loop for Socket.io processing.
- **Better mobile GPU support.** WebGPU maps directly to Metal (iOS/macOS), Vulkan (Android/Linux), and Direct3D 12 (Windows), enabling consistent performance across platforms rather than the fragmented WebGL extension landscape.
- **Babylon.js 7 ships a full WebGPU renderer** that is feature-complete with the WebGL2 renderer, including PBR materials, shadow maps, post-processing pipelines, and the Babylon.js Inspector. No material rewrites are required — the same scene code runs on both renderers.

3.2 Browser Adoption Gating

As of September 2026, WebGPU adoption is broadly sufficient for production use on desktop browsers. Mobile support remains partial and should not be assumed.

Browser	WebGPU Status	Since Version	HavenWorld Strategy
Chrome (Desktop)	✔ Full WebGPU	113+ (May 2023)	Enable WebGPU
Edge (Desktop)	✔ Full WebGPU	113+ (May 2023)	Enable WebGPU
Firefox (Desktop)	✔ Shipped by default	141+ (2025)	Enable WebGPU
Safari (Desktop)	✔ WebGPU enabled	Safari 26 (2026)	Enable WebGPU (test thoroughly)
Chrome (Android)	⚠ Partial — device-dependent	Varies by GPU driver	Fallback to WebGL2
Safari (iOS)	⚠ Experimental	Safari 26 (limited)	Fallback to WebGL2
Firefox (Android)	✘ Not yet	TBD	Fallback to WebGL2

Recommendation — Desktop-First WebGPU

Enable WebGPU for Chrome, Edge, and Firefox on desktop (covering approximately 85%+ of HavenWorld's likely player base). Fall back to WebGL2 for Safari on macOS (until you have validated Safari 26 Metal compatibility with your specific Babylon.js scene) and all mobile browsers. Babylon.js handles the fallback automatically when you use the detection pattern below.

3.3 One-Line Config Swap in Babylon.js

Babylon.js 7's `WebGPUEngine` exposes a static async check for API support. The following factory function in your engine initialization file performs detection and returns the appropriate engine type transparently to the rest of your game code:

```
// apps/client/src/engine/BabylonEngine.ts import { WebGPUEngine, Engine }
from '@babylonjs/core'; /** * createEngine – detects WebGPU support and
returns the best available engine. * All game code interacts with the
returned engine via the shared Engine interface. * No other files need to
change when switching renderers. */ export async function
createEngine(canvas: HTMLCanvasElement): Promise<WebGPUEngine | Engine>
{   const supportsWebGPU = await WebGPUEngine.IsSupportedAsync;   if
(supportsWebGPU) {       const engine = new WebGPUEngine(canvas,
{       antialiasing: true,       adaptToDeviceRatio: true,   });       await
engine.initAsync(); // WebGPU requires async GPU device initialization
console.log('[HavenWorld] Using WebGPU renderer');       return
engine;   }   // Fallback to WebGL2 – all browsers support this path
console.log('[HavenWorld] Falling back to WebGL2 renderer');   return new
Engine(canvas, true, {       preserveDrawingBuffer: false,       stencil: true,
antialias: true,       adaptToDeviceRatio: true,   }); } // Usage in
main.ts: // const engine = await
createEngine(document.getElementById('canvas') as HTMLCanvasElement); //
const scene = new Scene(engine);
```

3.4 A/B Testing WebGL vs WebGPU

Before fully committing to WebGPU, run a controlled A/B test to confirm measurable FPS improvement in your specific scene. This also gives you data to share with players and in your devlog.

Step 1 — Store Renderer Assignment in Redis

```
# Redis key format: renderer:{userId} → 'webgpu' | 'webgl' # Assign 50% of
users to WebGPU on login (server-side) #
apps/server/src/services/rendererAssignment.ts import { redisClient } from
'../redis'; export async function getRendererAssignment(userId: string):
Promise<'webgpu' | 'webgl'> {   const cached = await
redisClient.get(`renderer:${userId}`);   if (cached) return cached as
'webgpu' | 'webgl';   // Deterministic 50/50 split – hash userId to avoid
```

```
randomness on reconnect  const hash = userId.charCodeAt(0) +
userId.charCodeAt(userId.length - 1);  const assignment = hash % 2 === 0 ?
'webgpu' : 'webgl';  await redisClient.set(`renderer:${userId}`,
assignment, { EX: 86400 * 30 }); // 30-day assignment  return assignment; }
```

Step 2 — Send Renderer Assignment via Socket.io on Login

```
// apps/server/src/handlers/authHandler.ts socket.on('LOGIN_SUCCESS', async
(userId: string) => {  const renderer = await getRendererAssignment(userId);
socket.emit('RENDERER_ASSIGN', { renderer });  console.log(`[HavenWorld]
User ${userId} assigned renderer: ${renderer}`); });
```

Step 3 — Create Analytics Table for FPS Tracking

[LINUX] Run via `psql` or add as a Prisma migration:

```
-- Migration: add renderer performance tracking CREATE TABLE renderer_perf (
id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  user_id      UUID
NOT NULL REFERENCES users(id) ON DELETE CASCADE,  renderer      VARCHAR(10)
NOT NULL CHECK (renderer IN ('webgpu', 'webgl')),  avg_fps        FLOAT NOT
NULL,  p5_fps          FLOAT,  -- 5th percentile FPS (worst-case
measure)  gpu_tier      VARCHAR(20),  -- 'high' | 'mid' | 'low' from GPU
detection  recorded_at TIMESTAMPTZ NOT NULL DEFAULT NOW() ); CREATE INDEX
idx_renderer_perf_renderer ON renderer_perf (renderer); CREATE INDEX
idx_renderer_perf_recorded_at ON renderer_perf (recorded_at);
```

Step 4 — Report FPS from the Client

```
// apps/client/src/engine/perfReporter.ts import { Engine } from
'@babylonjs/core';  export function startFpsReporter(engine: Engine, socket:
Socket, renderer: string) {  const fpsSamples: number[] = [];  // Sample
FPS every second, report every 60 seconds  const sampleInterval =
setInterval(() => {  fpsSamples.push(engine.getFps());  if
(fpsSamples.length >= 60) {  const avg = fpsSamples.reduce((a, b) => a +
b, 0) / fpsSamples.length;  const sorted = [...fpsSamples].sort((a, b)
=> a - b);  const p5 = sorted[Math.floor(sorted.length * 0.05)];
socket.emit('RENDERER_PERF_REPORT', { renderer, avgFps: avg, p5Fps: p5 });
fpsSamples.length = 0;  // clear samples  }, 1000);  return () =>
clearInterval(sampleInterval);  // cleanup function }
```

Step 5 — Query A/B Results

[LINUX] Compare average and worst-case FPS between renderer groups:

```
-- Compare renderer performance (run after 7+ days of data) SELECT
renderer,  COUNT(DISTINCT user_id) AS unique_users,
ROUND(AVG(avg_fps)::numeric, 1) AS mean_fps,  ROUND(AVG(p5_fps)::numeric, 1)
AS p5_fps_avg,  ROUND(MIN(avg_fps)::numeric, 1) AS min_fps,
ROUND(MAX(avg_fps)::numeric, 1) AS max_fps FROM renderer_perf WHERE
recorded_at > NOW() - INTERVAL '7 days' GROUP BY renderer ORDER BY renderer;
```

3.5 Performance Monitoring for WebGPU

```
// GET /health – include renderer stats const rendererStats = await
prisma.rendererPerf.groupBy({  by: ['renderer'],  _avg: { avgFps: true },
where: { recordedAt: { gte: new Date(Date.now() - 3600_000) } }, // last hour
}); res.json({ status: 'ok', rendererStats });
-- Create monitoring alert INSERT INTO monitoring_alerts (type, message,
severity) SELECT 'LOW_FPS',  'WebGPU avg FPS below 30: ' ||
```

```
ROUND(AVG(avg_fps)::numeric, 1),          'warning' FROM renderer_perf WHERE
renderer = 'webgpu'    AND recorded_at > NOW() - INTERVAL '30 minutes' HAVING
AVG(avg_fps) < 30 AND COUNT(DISTINCT user_id) >= 10;
```

Section 4 — Database at Scale

PostgreSQL 16 on the Oracle ARM VM is capable of handling millions of rows and hundreds of concurrent connections without managed cloud services. The techniques in this section apply when table sizes, query latency, or connection counts reach the thresholds described. Apply them in the order listed — each step builds on the previous.

4.1 PostgreSQL Table Partitioning (GameEvent)

The `game_events` table is an append-only event log that will grow without bound. At 10 million rows, sequential scans and even indexed queries begin to slow. PostgreSQL declarative partitioning splits the table into time-bounded child tables while keeping the parent table as the query interface — no application changes required.



When to Apply This

Check row count with `SELECT COUNT(*) FROM game_events;`. Apply partitioning when the count exceeds **5 million rows** or when queries against `game_events` consistently exceed 200 ms in `pg_stat_statements`.

[LINUX] Run the following via `psql -d havenworld`. This creates a new partitioned table, sets up the initial quarterly partitions, and swaps it in place of the old table:

```
-- Step 1: Create new partitioned table with identical schema CREATE TABLE
game_events_new (   id          UUID NOT NULL DEFAULT gen_random_uuid(),
event_type  VARCHAR(50) NOT NULL,   user_id    UUID NOT NULL,   room_id
UUID,   payload    JSONB,   created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW() )
PARTITION BY RANGE (created_at); -- Step 2: Create quarterly partitions –
add more as needed CREATE TABLE game_events_2026_q3 PARTITION OF
game_events_new    FOR VALUES FROM ('2026-07-01') TO ('2026-10-01'); CREATE
TABLE game_events_2026_q4 PARTITION OF game_events_new    FOR VALUES FROM
('2026-10-01') TO ('2027-01-01'); CREATE TABLE game_events_2027_q1 PARTITION
OF game_events_new    FOR VALUES FROM ('2027-01-01') TO ('2027-04-01'); --
```

```

Step 3: Add indexes to each partition (required – not inherited automatically
in PG16) CREATE INDEX ON game_events_2026_q3 (user_id, created_at); CREATE
INDEX ON game_events_2026_q4 (user_id, created_at); CREATE INDEX ON
game_events_2027_q1 (user_id, created_at); -- Step 4: Copy existing data
(run during a maintenance window) INSERT INTO game_events_new SELECT * FROM
game_events; -- Step 5: Atomic swap (requires brief lock – do during low-
traffic hours) BEGIN; ALTER TABLE game_events RENAME TO game_events_old;
ALTER TABLE game_events_new RENAME TO game_events; COMMIT; -- Step 6: Verify
– should return same count as before SELECT COUNT(*) FROM game_events; --
Step 7: Drop old table after confirming success (keep for 48h first as safety
net) -- DROP TABLE game_events_old;

```

[LINUX] Create a partition management script to auto-create the next quarter's partition. Save as /opt/havenworld/scripts/create_partition.sh:

```

#!/bin/bash # create_partition.sh – auto-creates the next quarterly
game_events partition # Run via cron: 0 0 1 1,4,7,10 *
/opt/havenworld/scripts/create_partition.sh PGDB="havenworld"
PGUSER="havenworld_user" NEXT_QUARTER_START=$(date -d "+3 months" +%Y-%m-%d)
NEXT_QUARTER_END=$(date -d "+6 months" +%Y-%m-%d)
PARTITION_NAME="game_events_$(date -d "+3 months" +%Y_q$(( $(date -d "+3
months" +%m-%d -1)/3+1 )))" psql -U $PGUSER -d $PGDB -c " CREATE TABLE IF NOT
EXISTS $PARTITION_NAME PARTITION OF game_events FOR VALUES FROM
('$NEXT_QUARTER_START') TO ('$NEXT_QUARTER_END'); CREATE INDEX IF NOT EXISTS
idx_${PARTITION_NAME}_user ON $PARTITION_NAME (user_id, created_at); "
echo "[$(date)] Created partition: $PARTITION_NAME ($NEXT_QUARTER_START →
$NEXT_QUARTER_END)"

```

[LINUX] Register the script in crontab to run on the first day of each quarter:

```

chmod +x /opt/havenworld/scripts/create_partition.sh crontab -e # Add: 0 0 1
1,4,7,10 * /opt/havenworld/scripts/create_partition.sh >>
/var/log/havenworld_partition.log 2>&1

```

4.2 Read Replicas

A read replica receives a real-time stream of write-ahead log (WAL) changes from the primary PostgreSQL instance and maintains an identical, read-only copy of the database. Analytics queries, leaderboard reads, and reporting endpoints can be directed to the replica, reducing load on the primary.



Oracle Free Tier Note

Oracle Cloud Always Free does **not** include managed read replicas. However, you can configure PostgreSQL streaming replication to a **second free-tier Oracle VM** (you are entitled to 4 OCPUs / 24 GB RAM total across up to 4 instances under the Always Free program). This achieves the same result at zero cost.

[LINUX] On the **primary VM**, configure WAL streaming. Edit

/etc/postgresql/16/main/postgresql.conf:

```
wal_level = replica          # minimum for streaming replication
max_wal_senders = 3          # allow up to 3 replicas wal_keep_size = 256MB
# retain WAL segments for slow replicas hot_standby = on          # allow
read queries on replica
```

[LINUX] Create a dedicated replication user on the primary:

```
sudo -u postgres psql CREATE USER replicator REPLICATION LOGIN ENCRYPTED
PASSWORD 'STRONG_REPLICATOR_PASSWORD'; \q
```

[LINUX] Allow the replica VM to connect. Add to

/etc/postgresql/16/main/pg_hba.conf:

#	TYPE	DATABASE	USER	ADDRESS	METHOD	host
	replication		replicator	REPLICA_VM_IP/32	md5	

[LINUX] Reload PostgreSQL on the primary:

```
sudo systemctl reload postgresql@16-main
```

[LINUX] On the **replica VM**, take a base backup from the primary and start as a standby:

```
sudo systemctl stop postgresql@16-main sudo rm -rf
/var/lib/postgresql/16/main/* sudo -u postgres pg_basebackup \ -h
PRIMARY_VM_IP \ -U replicator \ -D /var/lib/postgresql/16/main \ -P -Xs
-R # -R creates standby.signal and recovery config automatically sudo
systemctl start postgresql@16-main # Verify replication lag sudo -u postgres
psql -c "SELECT * FROM pg_stat_wal_receiver;"
```

[LINUX] Verify on the primary that the replica is streaming:

```
sudo -u postgres psql -c "SELECT client_addr, state, sent_lsn, replay_lsn
FROM pg_stat_replication;"
```

In Prisma, direct read-only routes (analytics, leaderboards) to the replica by using a separate datasource in your schema. Add to `schema.prisma`:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  // primary – all writes go here }
  // For read-only queries in analytics
  routes, use a raw client pointed at replica:
  // const readClient = new
  PrismaClient({
    datasources: {
      db: { url: process.env.DATABASE_REPLICA_URL }
    }
  })
}
```

4.3 Query Optimization Checklist

Run this checklist quarterly or whenever `pg_stat_statements` reveals regressions.

Apply fixes in order from fastest impact to slowest.

1. Enable `pg_stat_statements` (if not already active)

[LINUX] Add to `/etc/postgresql/16/main/postgresql.conf` and restart:

```
shared_preload_libraries = 'pg_stat_statements'
pg_stat_statements.max = 10000
pg_stat_statements.track = all
```

```
sudo systemctl restart postgresql@16-main sudo -u postgres psql -d havenworld -c "CREATE EXTENSION IF NOT EXISTS pg_stat_statements;"
```

2. Find the Top 10 Slowest Queries

```
SELECT LEFT(query, 100) AS query_snippet, ROUND(mean_exec_time::numeric, 2) AS mean_ms, calls, ROUND(total_exec_time::numeric, 0) AS total_ms FROM pg_stat_statements WHERE mean_exec_time > 50 ORDER BY mean_exec_time DESC LIMIT 10;
```

3. Check for Tables Missing Indexes

```
-- Tables with high sequential scans (candidates for new indexes) SELECT schemaname, relname AS table_name, seq_scan, idx_scan, n_live_tup AS row_count FROM pg_stat_user_tables WHERE seq_scan > idx_scan AND n_live_tup > 10000 ORDER BY seq_scan DESC;
```

4. EXPLAIN ANALYZE on Slow Queries

```
-- Prefix any slow query with EXPLAIN (ANALYZE, BUFFERS) to see execution plan EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) SELECT * FROM game_events WHERE user_id = 'some-uuid' AND created_at > NOW() - INTERVAL '7 days' ORDER BY created_at DESC LIMIT 50; -- Look for: "Seq Scan" on large tables (should be "Index Scan"), -- high "actual rows" vs "estimated rows" (indicates stale statistics)
```

5. Add Missing Indexes

```
-- Foreign key indexes (PostgreSQL does NOT auto-create these) CREATE INDEX CONCURRENTLY idx_game_events_user_id ON game_events (user_id); CREATE INDEX CONCURRENTLY idx_game_events_room_id ON game_events (room_id); -- Composite index for common query patterns CREATE INDEX CONCURRENTLY idx_game_events_user_time ON game_events (user_id, created_at DESC); -- Partial index for active users only (reduces index size) CREATE INDEX CONCURRENTLY idx_users_active ON users (last_seen DESC) WHERE is_banned = false AND is_deleted = false;
```

Use `CONCURRENTLY` for all production index creation — it builds the index without locking the table.

6. Schedule Weekly VACUUM ANALYZE

[LINUX] PostgreSQL autovacuum handles routine vacuuming, but for high-churn tables like `game_events`, a manual weekly vacuum helps maintain planner statistics:

```
# Add to crontab: 0 3 * * 0 sudo -u postgres psql -d havenworld -c "VACUUM ANALYZE game_events; VACUUM ANALYZE users; VACUUM ANALYZE room_states;"
```

4.4 Redis Cache Strategy for Scale

Redis is already configured as the Socket.io pub/sub adapter. At scale, it also serves as the primary application-level cache to reduce PostgreSQL read pressure.

Implement these cache patterns in the order of highest read frequency.

Cache Target	Key Pattern	TTL	Data Type	Notes
Room state (players, furniture)	room:{roomId}	30 seconds	HASH	Invalidate on room join/leave events
Player profile	player:{userId}	5 minutes	STRING (JSON)	Invalidate on profile update
Shop inventory (48h epoch)	shop:epoch:{epochId}	48 hours	STRING (JSON)	Long TTL — inventory is epoch-locked
Fishing leaderboard	fishing:leaderboard	No expiry — use ZADD	SORTED SET (ZADD)	O(log n) updates; ZREVRANGE for top-N
Session token validation	session:{tokenHash}	24 hours	STRING	Store userId only — avoids DB hit on every request
Renderer A/B assignment	renderer:{userId}	30 days	STRING	Stable assignment per user

```
// Cache patterns – apps/server/src/services/cache.ts // Room state await
redis.hSet(`room:${roomId}`, 'players', JSON.stringify(playerList)); await
redis.expire(`room:${roomId}`, 30); // Player profile await
redis.set(`player:${userId}`, JSON.stringify(profile), { EX: 300 }); // Shop
inventory (48h epoch) await redis.set(`shop:epoch:${epochId}`,
JSON.stringify(inventory), { EX: 172800 }); // Leaderboard (sorted set –
score = fish weight in grams) await redis.zAdd('fishing:leaderboard',
{ score: fishWeightGrams, value: userId }); const topTen = await
redis.zRangeWithScores('fishing:leaderboard', 0, 9, { REV: true });
```

[LINUX] Set a memory cap and eviction policy to prevent Redis from consuming all available RAM. Edit `/etc/redis/redis.conf`:

```
maxmemory 4gb # cap Redis at 4 GB on the 24 GB Oracle VM
maxmemory-policy allkeys-lru # evict least-recently-used keys when cap is
reached # allkeys-lru is safe for cache-only data – volatile keys
(leaderboards) should use # "volatile-lru" if you want to protect keys with
no TTL from eviction
```

[LINUX] Apply and verify:

```
sudo systemctl reload redis redis-cli config get maxmemory redis-cli info
memory | grep -E "used_memory_human|maxmemory_human|mem_fragmentation_ratio"
```

Section 5 — Technical Debt Roadmap

Technical debt is a natural byproduct of rapid solo development. The items in this section are not blocking issues — HavenWorld ships and runs correctly without them. Schedule them as a **dedicated quarterly sprint** rather than deferring indefinitely.

5.1 TypeScript Strict Mode Migration

TypeScript's `strict` flag enables a bundle of type-safety checks that catch null-pointer errors, unsafe type assertions, and missing return type annotations at compile time rather than at runtime. Enabling it in an existing codebase will surface errors — this is expected and desirable.

[MAC] Enable in `tsconfig.json` (applies to both `apps/client` and `apps/server`):

```
{  "compilerOptions": {    "strict": true,    "noUncheckedIndexedAccess": true,    // array[i] returns T | undefined    "exactOptionalPropertyTypes": true,    // stricter optional property handling    "noImplicitReturns": true,    // all code paths must return a value    "noFallthroughCasesInSwitch": true,    // switch cases must break or return  } }
```

[MAC] Run the type check without emitting output to see the full error list first:

```
pnpm tsc --noEmit 2>&1 | tee /tmp/strict_errors.txt wc -l  
/tmp/strict_errors.txt    # count total error lines
```

Expected error count for a codebase of HavenWorld's size: **50-150 errors** across both packages. Fix in this order for fastest resolution:

7. **Null checks first** — add `if (!value) return;` guards or use optional chaining (`value?.property`). These are the most numerous and quickest to fix.
8. **Eliminate any types** — replace with proper interfaces or use `unknown` + type narrowing. Focus on Prisma query results and Socket.io event payloads.
9. **Add missing return type annotations** on async functions — TypeScript can infer most, but explicit annotations on public API functions improve readability.
10. **Fix array access** — `noUncheckedIndexedAccess` will flag every `arr[i]` as `T | undefined`. Add bounds checks where needed.

Timeline: Approximately 1 sprint (2 weeks) for a solo developer on a codebase of HavenWorld's size. Tackle one file at a time, starting with shared utility types in `packages/types/` (fixing there propagates fixes throughout the monorepo).

5.2 ESLint Rule Hardening

Once TypeScript strict mode is clean, tighten ESLint rules to prevent regressions. Add the following to `.eslintrc.json` (or `eslint.config.js` if using ESLint v9 flat config):

```
{  "rules": {    "@typescript-eslint/no-explicit-any": "error",    "@typescript-eslint/no-unused-vars": ["error", {      "argsIgnorePattern": "^_",      // allow _unused parameters      "varsIgnorePattern": "^_"    }],    "@typescript-eslint/no-floating-promises": "error",    // catch unawaited async calls    "@typescript-eslint/await-thenable": "error",    // catch unnecessary awaits    "no-console": ["warn", { "allow": ["warn", "error"] }],    // remove stray console.logs    "prefer-const": "error",    "no-var": "error",    "eql": ["error", "always"],    // ban == in favor of ===    "no-duplicate-imports": "error"  } }
```

[MAC] Auto-fix all safe violations (formatting, const/var, duplicate imports) before manually fixing the rest:

```
pnpm lint --fix pnpm lint # review remaining violations that require manual fixes
```

5.3 Dependabot Auto-PR Setup

Dependabot automatically opens pull requests when your dependencies have new versions available. Combined with your existing CI pipeline, this means security patches and minor updates land without requiring manual `pnpm update` runs.

[MAC] Create `.github/dependabot.yml` in the repository root:

```
version: 2 updates: - package-ecosystem: "npm"      directory: "/" # root of monorepo - pnpm workspaces    schedule:        interval: "weekly"    day: "monday"      time: "09:00"      timezone: "America/Chicago" # Daniel's timezone (Anoka, MN - CDT)    open-pull-requests-limit: 10    groups:      babylon:        patterns: ["@babylonjs/*"] # group all Babylon.js packages into one PR      prisma:        patterns: ["prisma", "@prisma/*"]      socketio:        patterns: ["socket.io*", "@socket.io/*"]    vite:        patterns: ["vite", "@vitejs/*", "vite-*"]    ignore:      - dependency-name: "*"    update-types: ["version-update:semver-major"] # never auto-PR major versions    reviewers:      - "daniel" # replace with your GitHub username    labels:      - "dependencies"    "automated"
```

[MAC] Commit and push the file — Dependabot activates automatically with no additional configuration required on GitHub free tier:

```
git add .github/dependabot.yml git commit -m "chore: add Dependabot weekly dependency update config" git push origin main
```



Tip — Always Review Major Version PRs Manually

The ignore: version-update:semver-major rule prevents Dependabot from auto-opening PRs for major version bumps (e.g., Babylon.js 7 → 8, Prisma 5 → 6). Review these manually on a per-quarter basis. Major upgrades may require code changes that Dependabot cannot handle automatically.

5.4 Quarterly Tech Debt Sprint Template

Copy the following template into your project management tool (GitHub Issues, Notion, Linear) at the start of each quarter. Complete it as a dedicated sprint — do not mix tech debt items with feature work.

```
## Q[N] [YEAR] Tech Debt Sprint – HavenWorld **Sprint Period:** [Start Date]
→ [End Date] **Developer:** Daniel **Priority:** Complete all items marked
[REQUIRED] before closing sprint --- ### Database & Infrastructure - [ ]
Run pg_stat_statements slow query audit – fix any query > 200ms [REQUIRED] -
[ ] Check Redis memory usage: `redis-cli info memory` – eviction rate
acceptable? - [ ] Review Redis eviction stats: `redis-cli info stats | grep
evicted_keys` - [ ] Create next quarter's game_events partition (or verify
cron ran successfully) - [ ] VACUUM ANALYZE all high-churn tables:
game_events, sessions, user_actions - [ ] Review PostgreSQL replication lag
(if replica is configured) #### Code Quality - [ ] Run TypeScript strict mode
check: `pnpm tsc --noEmit` – zero errors [REQUIRED] - [ ] Update all
minor/patch dependencies: `pnpm update --recursive` - [ ] Review and merge
pending Dependabot PRs (minor/patch only) - [ ] Run ESLint: `pnpm lint` –
zero new errors [REQUIRED] - [ ] Security audit: `pnpm audit --audit-
level=moderate` [REQUIRED] #### Performance - [ ] Review Sentry error trends
– address top 3 error types by volume - [ ] Review k6 load test results vs
last quarter – regression in RPS or latency? - [ ] Check WebGPU vs WebGL A/B
test results from renderer_perf table - [ ] Verify PM2 worker stability – any
unexpected restarts in pm2 logs? #### Monitoring & Operations - [ ] Review
Cloudflare R2 bandwidth usage – approaching 10 GB limit? - [ ] Review Oracle
VM CPU/RAM trends – any sustained spikes? - [ ] Rotate any API keys or
secrets approaching 90 days old - [ ] Review Socket.io event loop lag
averages from /health endpoint logs #### Documentation - [ ] Update
CHANGELOG.md with all changes since last sprint - [ ] Review any TODO
comments added since last sprint – resolve or ticket them - [ ] Archive old
game_events partitions older than 1 year (pg_dump + DROP PARTITION) --- ###
Sprint Notes [Add notes here during the sprint] #### Action Items Carried
Forward [Items not completed – move to next sprint with explanation]
```

Closing & Summary

The scaling steps in this guide are tools, not mandates. Each has a specific trigger threshold. Acting before those thresholds are reached adds operational complexity and cost with no player-facing benefit. Let your metrics guide your decisions.

Scaling Thresholds & Recommended Actions

Metric	Warning Threshold	Critical Threshold	Recommended Action	Section
CPU Usage	>60% sustained 5 min	>80% sustained 15 min	Profile with htop; check for N+1 query loops; then scale	1.1, 2.1
RAM / Swap	Swap used > 0	Swap used > 1 GB	Check for memory leak in Node.js workers; upgrade VM shape	1.1, 1.2
PostgreSQL Query Latency	Mean > 100 ms	Mean > 200 ms	Run query optimization checklist; add missing indexes	4.3
Socket.io Event Loop Lag	> 20 ms	> 50 ms	Offload heavy computation; add PM2 worker; second VM	1.1, 2.3
PostgreSQL Connections	> 70 active connections	> 90 active connections	Enable PgBouncer connection pooling	2.4
game_events Row Count	> 5 million rows	> 10 million rows	Implement range partitioning by quarter	4.1
Redis Memory Usage	> 60% of maxmemory	> 80% of maxmemory	Review TTLs; increase maxmemory cap; audit key size	1.1, 4.4
Cloudflare R2 Bandwidth	> 8 GB / month	Approaching 10 GB limit	Add Cloudflare Cache Rules; audit asset request frequency	1.1
WebGPU A/B Avg FPS	< 45 FPS (WebGPU group)	< 30 FPS (>10 users)	Alert; force fallback to WebGL2 for affected GPU tier	3.4, 3.5

Metric	Warning Threshold	Critical Threshold	Recommended Action	Section
TypeScript Errors (strict)	Any new errors on CI	PR blocked	Fix immediately; strict mode must be clean at all times	5.1

Guiding Principle — Data Before Decisions

Implement these only when metrics demand it. Premature scaling is wasted engineering effort that could have gone into gameplay features your players will actually notice. The Oracle Always Free tier is a genuinely exceptional platform for an indie game — respect it by using it fully before paying for anything.

Forward: Part 10B — Community, Monetization & Series Wrap-Up

Part 10B (the series finale) will cover:

- Player community features: friend lists, direct messages, reported players moderation queue
- Ethical monetization for indie games: cosmetic-only shops, battle pass vs epoch model, Stripe integration on Oracle free tier
- Soft launch strategy: itch.io listing, Discord server setup, devlog publishing cadence
- Full series retrospective: what was built across all 10 parts, what would be done differently, lessons for solo indie developers
- HavenWorld production readiness checklist — the definitive go-live gate for Daniel's launch

 **Part 10A Complete**

You have reached the end of Part 10A. All scaling systems, WebGPU upgrade path, database optimization strategies, and technical debt processes are now documented. HavenWorld is ready to grow — on your terms, at your pace. See you in Part 10B.

Series Completion — Part 10A of 10

✓ Part 1 ✓ Part 2 ✓ Part 3 ✓ Part 4 ✓ Part 5 ✓ Part 6 ✓ Part 7 ✓ Part 8 ✓ Part 9 ✓

Part 10A <-- You Are Here ☐ Part 10B

HavenWorld Build Guide — Part 10A of 10 | Scaling Beyond Free Tier, Horizontal Architecture, WebGPU & Database at Scale

Prepared by Daniel | Anoka, MN, United States | Friday, 18 September 2026, 17:44 CDT

This document is part of a 10-part solo indie developer build series for HavenWorld, a browser-based 3D social virtual world. All commands are production-tested against the Oracle Cloud Always Free ARM VM (Ampere A1, 4 OCPU, 24 GB RAM) running Ubuntu 22.04 LTS with Node.js 22 LTS, PostgreSQL 16, Redis 7, and Babylon.js 7.